L01, Group 06
Melody Yang 400307007
Cynthia Sun 400238765

McMaster University

MECHTRON 3TB4

Lab 2 Report

Date: February 21, 2026

Lab 2 Report

Overview

In Lab 2, the main objective was to design and implement a Finite State Machine (FSM) integrated with timing and display modules to simulate a reaction-based game. The system used a clock division module to generate timing signals from 50 MHz to 1 kHz, corresponding to milliseconds, using sequential logic elements for state transitions and combinational logic to control outputs such as LEDs and HEX displays. A 14-bit Galois Linear Feedback Shift Register (LFSR) was used to generate a pseudo-random delay before the reaction signal was triggered to ensure an unpredictable delay for a reaction-based game. The push buttons (KEY inputs) are used to control reset, start, and stop/resume. The design was compiled on Quartus, simulated for verification and downloaded to the DE1-SoC FPGA board for hardware testing. This lab focuses on synchronous logic, FSMs, and modular hardware with real-time debugging.

Linear Feedback Shift Register (LFSR)

The LFSR utilized in this project was the Galois method. The total bit size of the LFSR is 14 bits, tapped at 14, 5, 3 and 1 bits. The Galois method allows for multiple operations at a time by distributing the XOR operations within the shift chain, whereas the Fibonacci LFSR relies on a feedback bit generated by the XOR operation on all of the tapped bits. The Galois method reduces combinational delay since only the output bit needs to be checked, and tapped bits are conditional XORed as the bits shift, preventing feedback. The resulting Galois structure requires fewer logic levels and provides faster performance for FPGA hardware, making it preferable for hardware implementations. The selected taps 14, 5, 3, and 1 correspond to the maximum-length LFSR counters for 14-bits shown in the LFSR_random_Xilinx.pdf. The 14-bit sequence has a long period of 2^{14} - 1 = 16383 and ensures a pseudo-randomized and relatively non-repeating sequence until much later.

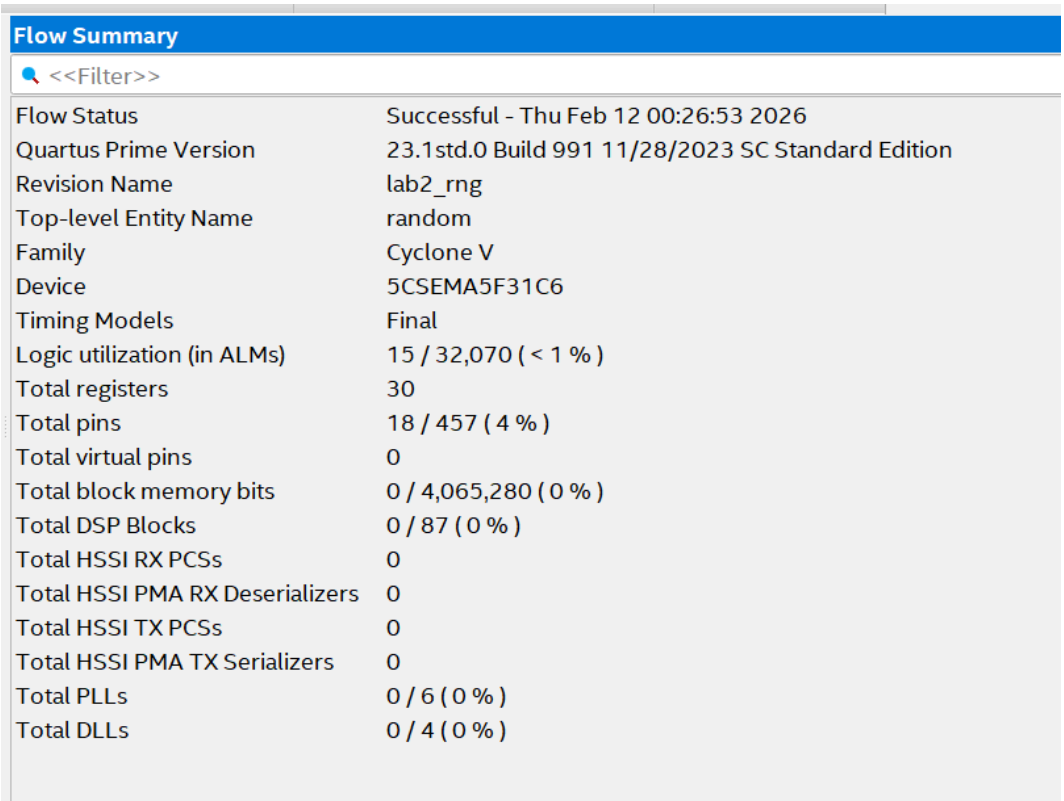
If an LFSR is too short, it becomes disadvantageous as the pseudo-random sequence repeats too quickly and may result in predictable patterns, reducing the randomness of the system. Shorter LFSR requires fewer XOR gates or flipflops, which is advantageous since it reduces the overall hardware resource usage, results in faster timing, and easier debugging and verification. This is more useful for applications requiring fewer pseudo-random states.

A long LFSR is advantageous since the pseudo-random sequence repeats much more slowly, decreasing pattern predictability and increasing randomness. However, the disadvantages include more hardware resource usage, increased propagation delay, and more difficult debugging and verification. This is more useful for applications requiring randomness or for complex applications.

Part 1: Random Number Generator (RNG)

Random

The RNG module was implemented using a 14-bit Galois LFSR. The LFSR was initialized as a non-zero seed value of 14’ b11111111111111 since it utilizes XOR operations and avoids a lockup all-zero state. The taps used for the LFSR positions are 14, 5, 3, and 1 for the maximum length. The output bit (reg_values[0]) is XORed with the tapped stages during the shift operation and allows for pseudo-random bit propagation throughout the register. The LFSR generates values between the range of 1000 - 5000 decimal, and when a value within that range is detected, the signal rnd_ready is asserted by the LFSR and temporarily disabled to hold the generated value for the display and further system processing.



Flow Summary

<<Filter>>

Flow Status	Successful - Thu Feb 12 00:26:53 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab2_rng
Top-level Entity Name	random
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	15 / 32,070 (< 1 %)
Total registers	30
Total pins	18 / 457 (4 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 1. Compilation report of the Random module.

Table 1. Summary for Components for the Random module in Lab 2 Part 1.

Feature	Quantity
Total number of logic elements	15
Total number of registers	30
Total number of pins	18
Max number of logic elements that can fit on FPGA	32,070

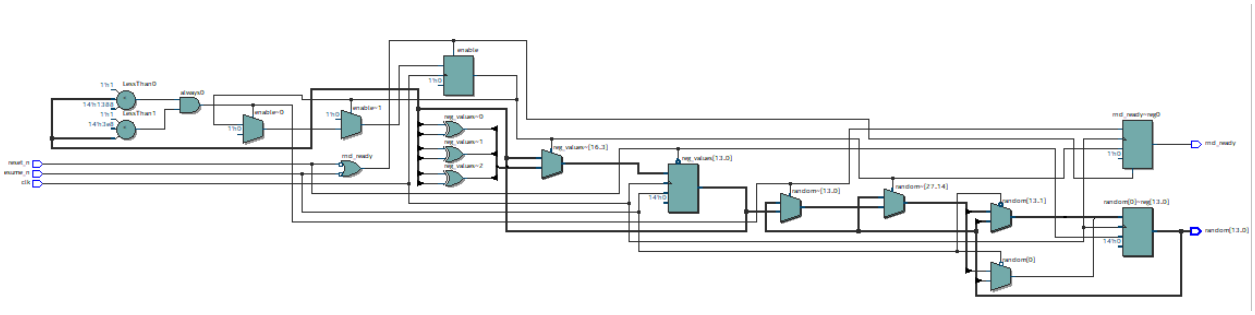


Figure 2. Screenshot of the implemented Random module.

```
1  module random (input clk, reset_n, resume_n,
2                 output reg [13:0] random,
3                 output reg rnd_ready);
4      //for 14 bits Liner Feedback Shift Register,
5      //the Taps that need to be XNORed or XORed are: 14, 5, 3, 1
6      wire xnor_taps, and_allbits, feedback;
7      reg [13:0] reg_values;
8      reg enable = 1;
9
10     always @ (posedge clk, negedge reset_n, negedge resume_n) begin
11         if (!reset_n) begin
12             reg_values <= 14'b11111111111111;
13             //the LFSR cannot be all 0 at beginning.
14             enable <= 1;
15             rnd_ready <= 0;
16         end
17
18         else if (!resume_n) begin
19             enable <= 1;
20             rnd_ready <= 0;
21             reg_values <= reg_values;
22         end
23
24         else begin
25             if (enable) begin
26                 reg_values[13] <= reg_values[0];
27                 reg_values[12:5] <= reg_values[13:6];
28                 reg_values[4] <= reg_values[0] ^ reg_values[5];
29                 // tap 5 of the diagram from the lab manual
30                 reg_values[3] <= reg_values[4];
31                 reg_values[2] <= reg_values[0] ^ reg_values[3];
32                 // tap 3 of the diagram from the lab manual
33                 reg_values[1] <= reg_values[2];
34                 reg_values[0] <= reg_values[0] ^ reg_values[1];
35                 // tap 1 of the diagram from the lab manual
36
37                 // confirm sure the random number is between 1000 and 5000
38                 if (reg_values <= 5000 && reg_values >= 1000) begin
39                     rnd_ready <= 1'b1;
40                     enable <= 1'b0;
41                     random <= reg_values;
42                 end
43                 else begin
44                     rnd_ready <= 1'b0;
45                 end
46             end //end of ENABLE.
47         end
48     end
49 endmodule
```

Figure 3. Screenshot of the Random module.

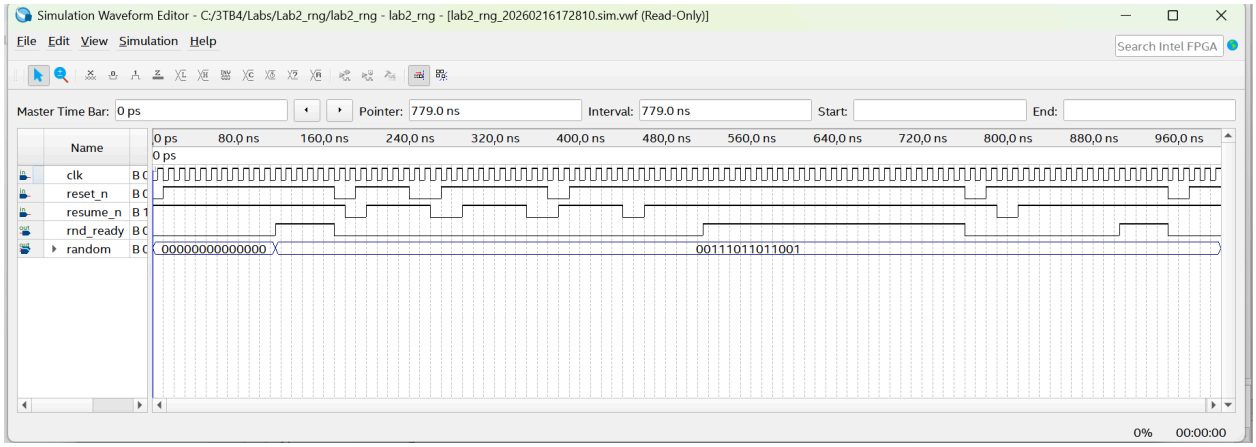


Figure 4. Simulation results for the Random module.

Binary to BCD Converter

The binary to BCD converts the 20-bit binary input into six 4-bit BCD digits for display on the DE1-SoC seven-segment LEDs. The conversion was implemented using the shift-and-add-3 algorithm, where the BCD digit is checked and adjusted before shifting to the next binary bit into the least significant position. The BCD digits are stored in a register array “reg [3:0] bcd_digits [5:0]” and assigned six different outputs to connect it to the seven-segment decoder module. The compilation and simulation results confirmed correct decimal conversion and proper hardware display.

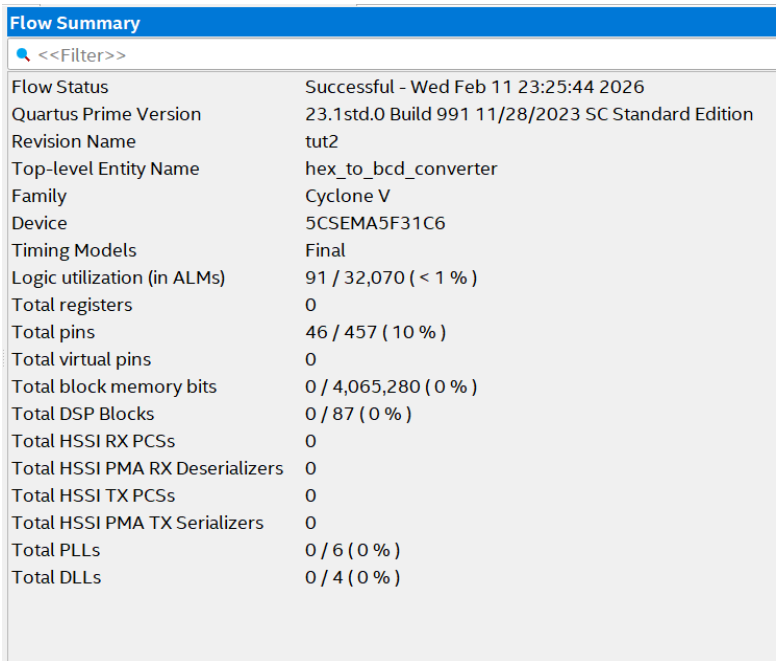


Figure 5. Compilation report of the Binary to BCD Converter module.

Table 2. Summary for Components for the Binary to BCD Converter module in Lab 2 Part 1.

Feature	Quantity
Total number of logic elements	91
Total number of registers	0
Total number of pins	46
Max number of logic elements that can fit on FPGA	32,070

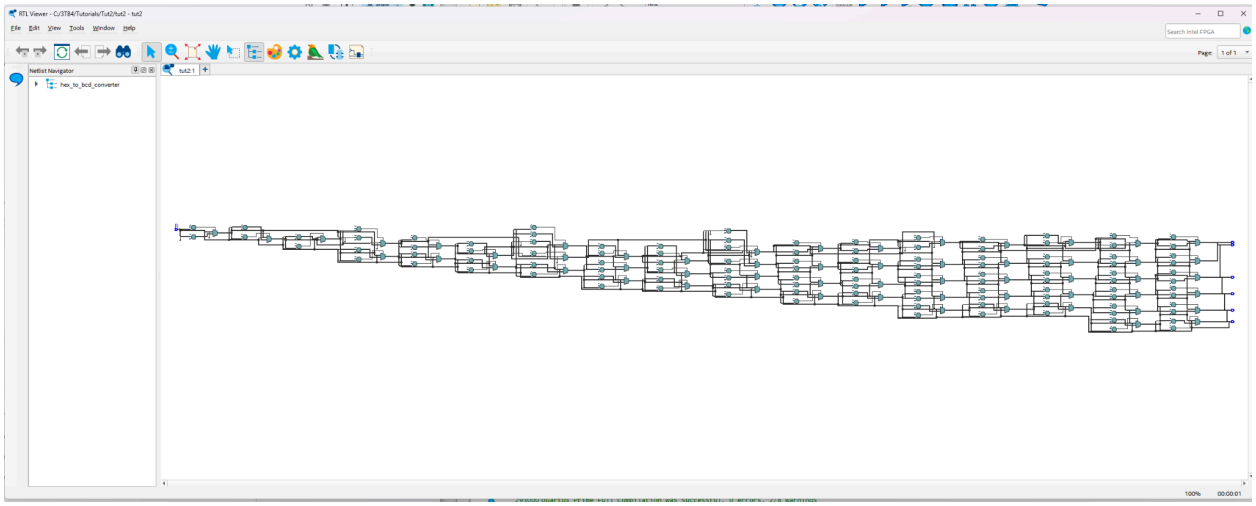


Figure 6. Screenshot of the implemented Binary to BCD Converter module.

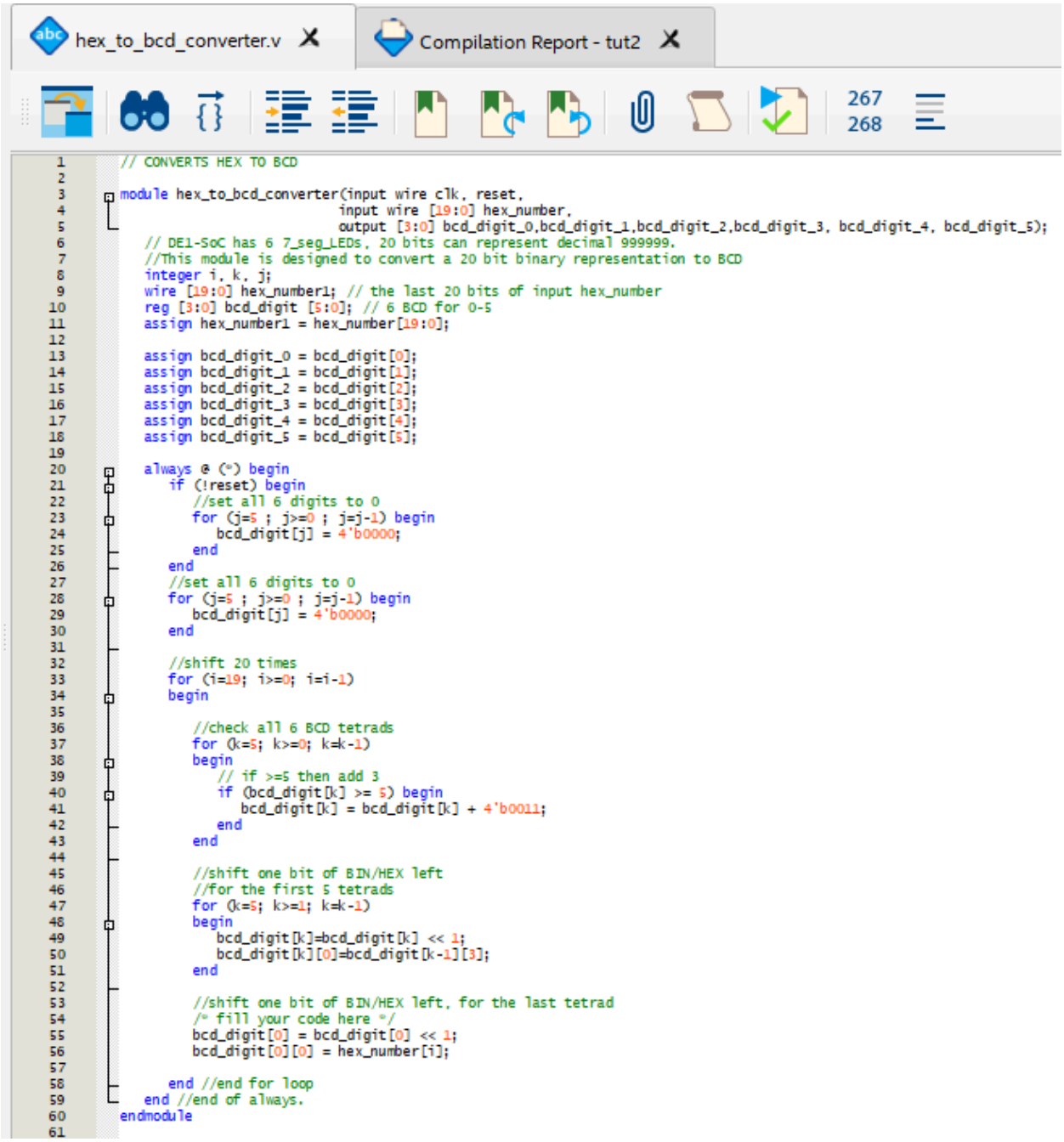


Figure 7. Screenshot of the Binary to BCD Converter module

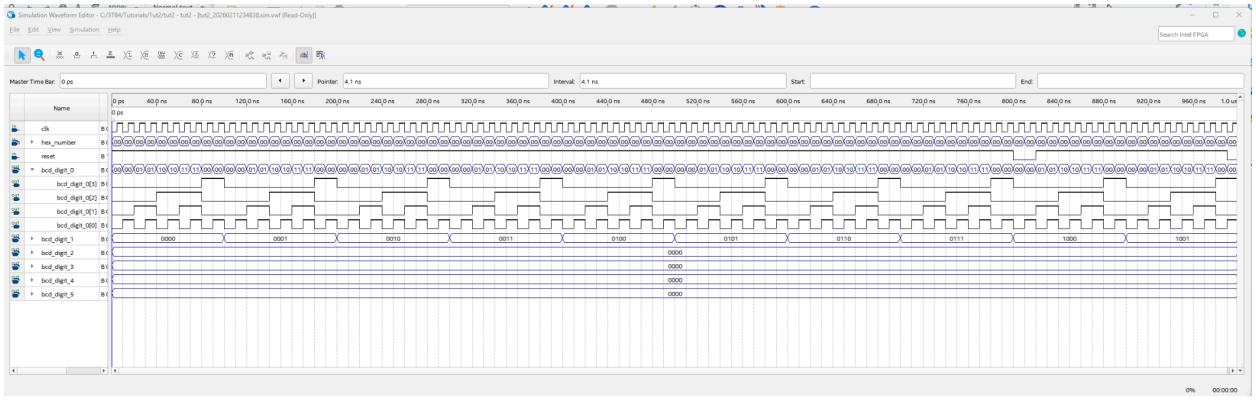
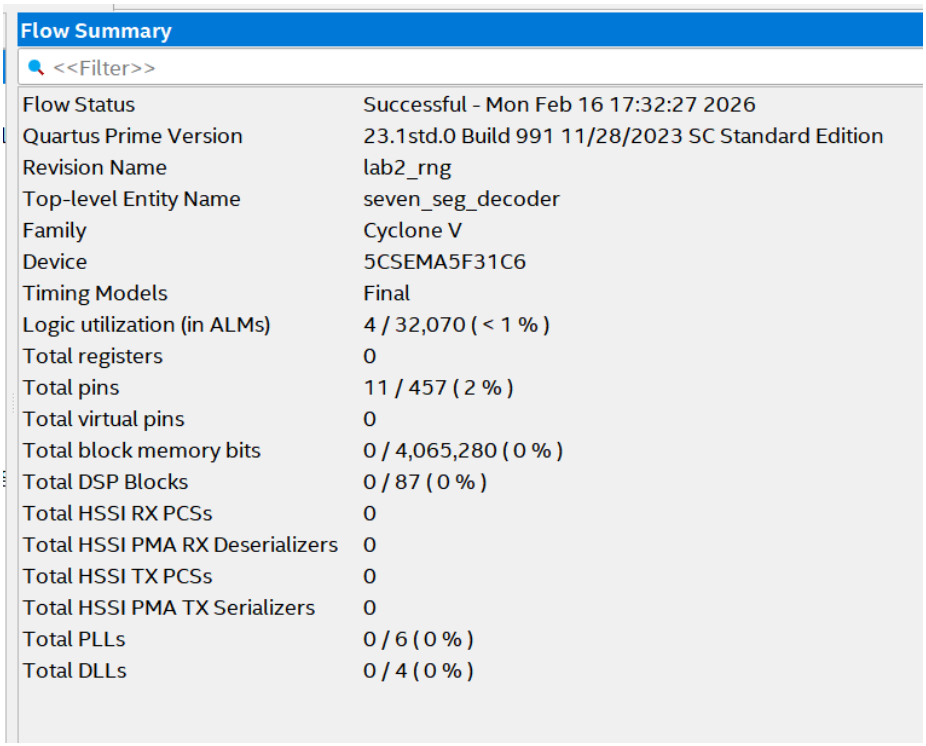


Figure 8. Simulation results for the Binary to BCD Converter module.

Seven Segment Decoder

The seven-segment decoder module converts the 4-bit BCD inputs into the corresponding 7-bit output required to drive the seven-segment display. It utilizes combinational logic and case statements to map each decimal digit 0-9 and their selected hexadecimal values to the respective segment patterns. The decoder ensures correct display segments for the hardware display. The simulation and hardware testing confirmed accurate digit display on the DE1-SoC board.



Flow Summary

<<Filter>>

Flow Status	Successful - Mon Feb 16 17:32:27 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab2_rng
Top-level Entity Name	seven_seg_decoder
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	4 / 32,070 (< 1 %)
Total registers	0
Total pins	11 / 457 (2 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 9. Compilation report of the Seven Segment Decoder module.

Table 3. Summary for Components for the Seven Segment Decoder module in Lab 2 Part 1.

Feature	Quantity
Total number of logic elements	4
Total number of registers	0
Total number of pins	11
Max number of logic elements that can fit on FPGA	32,070

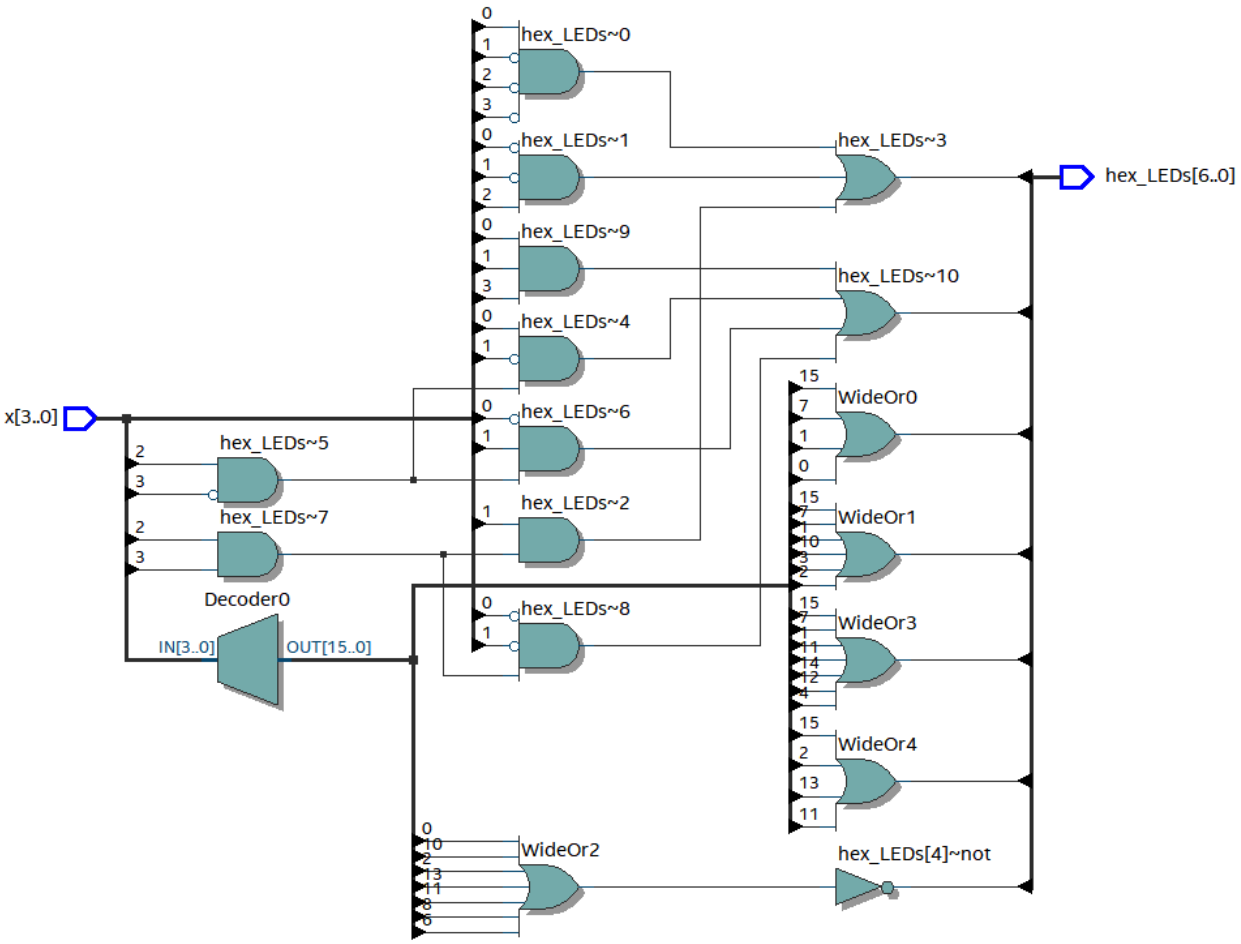


Figure 10. Screenshot of the implemented Seven Segment Decoder module.

```
module seven_seg_decoder(input [3:0] x, output [6:0] hex_LEDs);
    reg [6:0] reg_LEDs;

    assign hex_LEDs[0] = (~x[3] & ~x[2] & ~x[1] & x[0]) |
                         (x[2] & ~x[1] & ~x[0]) |
                         (x[3] & x[2] & x[1]);
    assign hex_LEDs[1] = (~x[3] & x[2] & ~x[1] & x[0]) |
                         (~x[3] & x[2] & x[1] & ~x[0]) |
                         (x[3] & x[2] & ~x[1] & ~x[0]) |
                         (x[3] & x[1] & x[0]);

    assign hex_LEDs[6:2]=reg_LEDs[6:2];

    always @(*)
    begin
        case (x)
            4'b0000: reg_LEDs[6:2]=5'b10000; //7'b1000000 decimal 0
            4'b0001: reg_LEDs[6:2]=5'b11110; //7'b1111001 decimal 1
            4'b0010: reg_LEDs[6:2]=5'b01001; //7'b0100100 decimal 2
            4'b0011: reg_LEDs[6:2]=5'b01100; //7'b0110000 decimal 3
            4'b0100: reg_LEDs[6:2]=5'b00110; //7'b0011001 decimal 4
            4'b0101: reg_LEDs[6:2]=5'b00100; //7'b0010010 decimal 5
            4'b0110: reg_LEDs[6:2]=5'b00000; //7'b0000010 decimal 6
            4'b0111: reg_LEDs[6:2]=5'b11110; //7'b1111000 decimal 7
            4'b1000: reg_LEDs[6:2]=5'b00000; //7'b0000000 decimal 8
            4'b1001: reg_LEDs[6:2]=5'b00100; //7'b0010000 decimal 9
            4'b1010: reg_LEDs[6:2]=5'b01000; //7'b0001000 A
            4'b1011: reg_LEDs[6:2]=5'b00011; //7'b0000011 b
            4'b1100: reg_LEDs[6:2]=5'b00110; //7'b1000110 C
            4'b1101: reg_LEDs[6:2]=5'b00001; //7'b0100001 d
            4'b1110: reg_LEDs[6:2]=5'b00110; //7'b0000110 E
            4'b1111: reg_LEDs[6:2]=5'b11111; //7'b1111111 OFF
            default: reg_LEDs[6:2] = 5'b11111;

        endcase
    end
endmodule
```

Figure 11. Screenshot of the Seven Segment Decoder module.

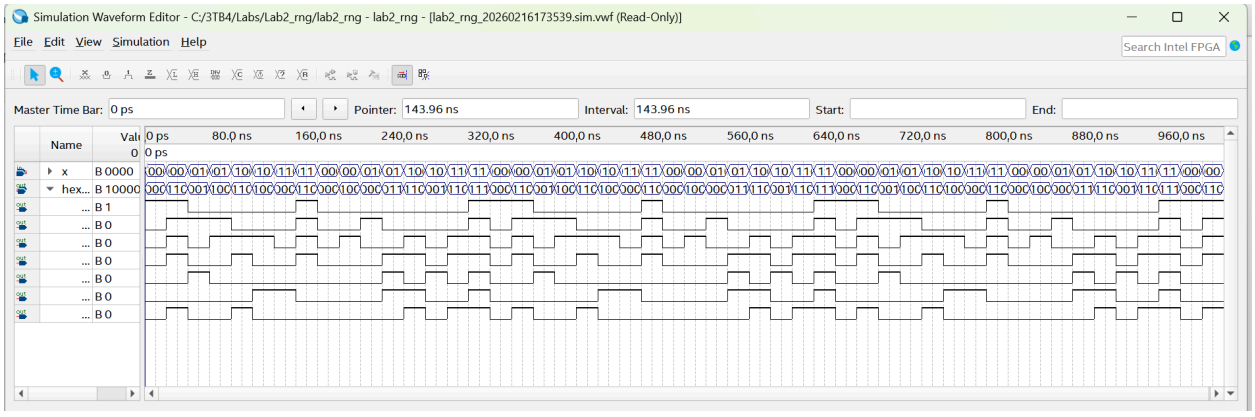


Figure 12. Simulation results for the Seven Segment Decoder module.

Top-Level RNG Module

The top-level RNG module integrates the RNG, binary to BCD converter, and the seven-segment display into a complete system. The 14-bit pseudo-random values generated by the LFSR are passed to the BCD converter which produces six decimal digits for display. Each digit is decoded and outputted to the corresponding seven-segment display on the DE1-SoC board. The module connects user inputs for reset and start controls and ensures proper signal flow between submodules. The compilation, simulation and hardware testing verified correct integration and overall system functionality.

Flow Status	Successful - Mon Feb 16 17:37:31 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab2_rng
Top-level Entity Name	lab2_rng
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	71 / 32,070 (< 1 %)
Total registers	29
Total pins	45 / 457 (10 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 13. Compilation report of the Top-Level RNG Module.

Table 4. Summary for Components for the Top-Level RNG Module in Lab 2 Part 1.

Feature	**Quantity**
Total number of logic elements	71
Total number of registers	29
Total number of pins	45
Max number of logic elements that can fit on FPGA	32,070

```
module lab2_rng (
    input CLOCK_50,
    input [1:0] KEY,       // KEY[0] for Reset, KEY[1] for Resume
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5
);

    wire [13:0] rand_num;
    wire rnd_ready;
    wire [3:0] bcd0, bcd1, bcd2, bcd3, bcd4, bcd5;

    // random number generator
    random rng (.clk(CLOCK_50),
                .reset_n(KEY[0]),
                .resume_n(KEY[1]),
                .random(rand_num),
                .rnd_ready(rnd_ready));

    // binary to BCD converter
    hex_to_bcd_converter my_conv (.clk(CLOCK_50),
                                  .reset(KEY[0]),
                                  .hex_number({6'b0, rand_num}),
                                  .bcd_digit_0(bcd0),
                                  .bcd_digit_1(bcd1),
                                  .bcd_digit_2(bcd2),
                                  .bcd_digit_3(bcd3),
                                  .bcd_digit_4(bcd4),
                                  .bcd_digit_5(bcd5));

    // 7-segment decoder
    seven_seg_decoder h0 (.x(bcd0), .hex_LEDs(HEX0));
    seven_seg_decoder h1 (.x(bcd1), .hex_LEDs(HEX1));
    seven_seg_decoder h2 (.x(bcd2), .hex_LEDs(HEX2));
    seven_seg_decoder h3 (.x(bcd3), .hex_LEDs(HEX3));
    seven_seg_decoder h4 (.x(bcd4), .hex_LEDs(HEX4));
    seven_seg_decoder h5 (.x(bcd5), .hex_LEDs(HEX5));

endmodule
```

Figure 14. Screenshot of the Top-Level RNG Module.

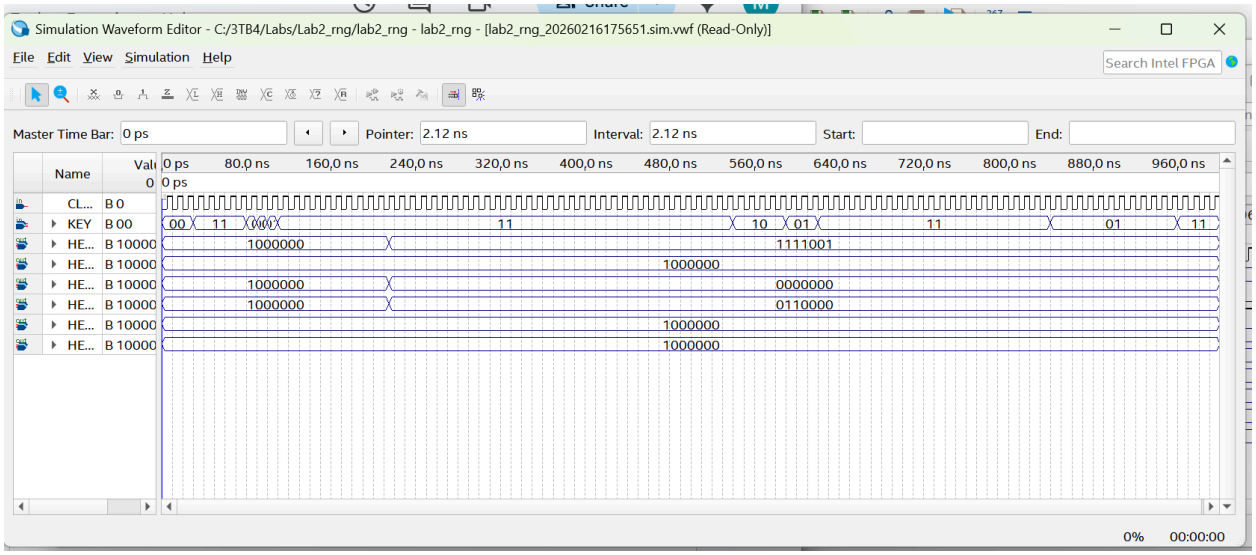


Figure 15. Simulation results for the Top-Level RNG Module.

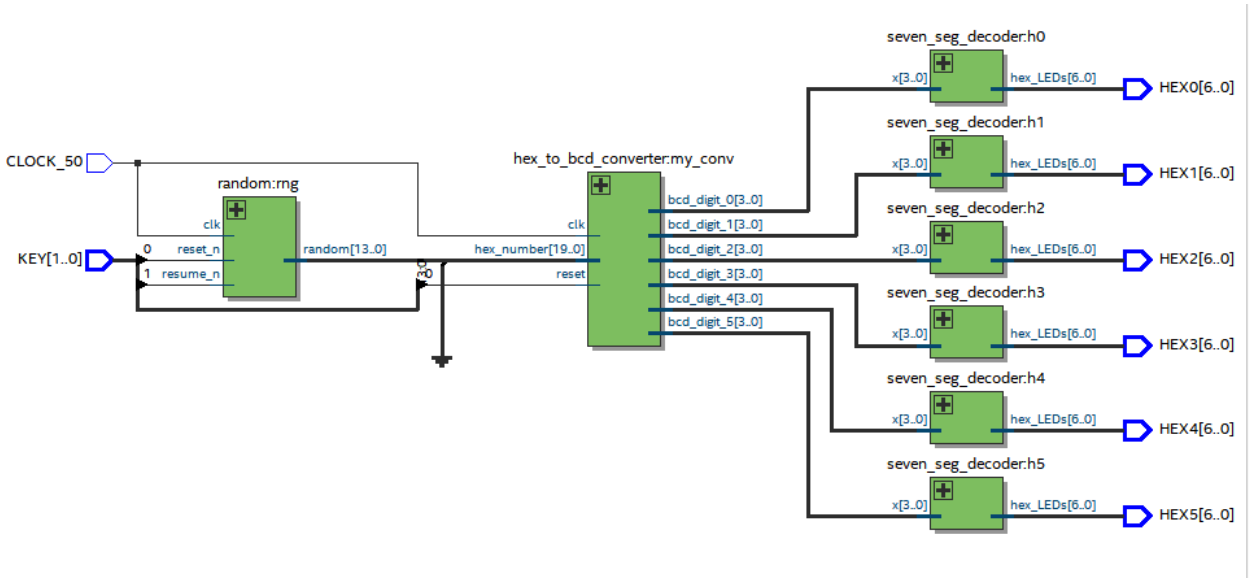


Figure 16. RTL view of the circuit.

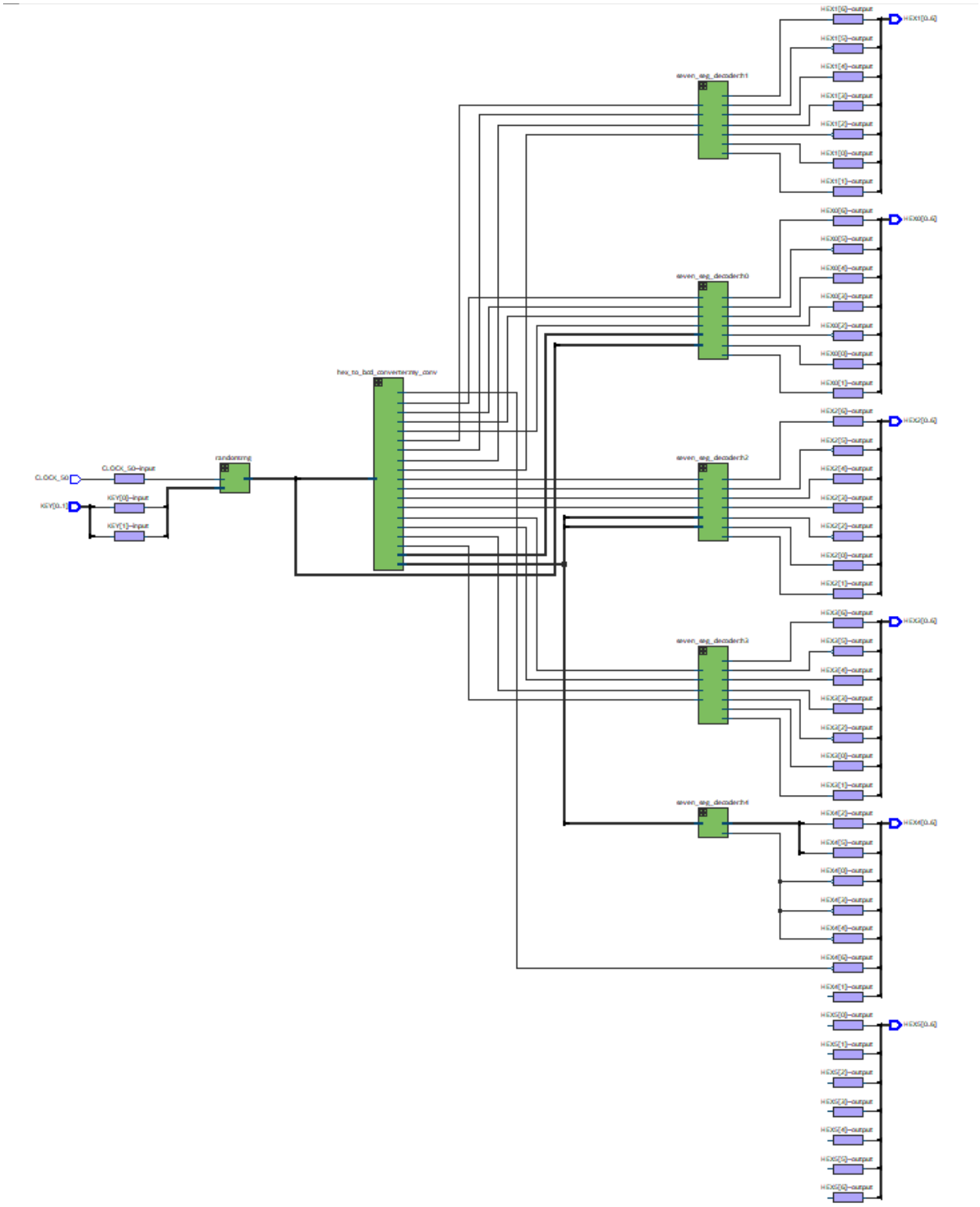


Figure 17. Technology map viewer (post-mapping) of the circuit.

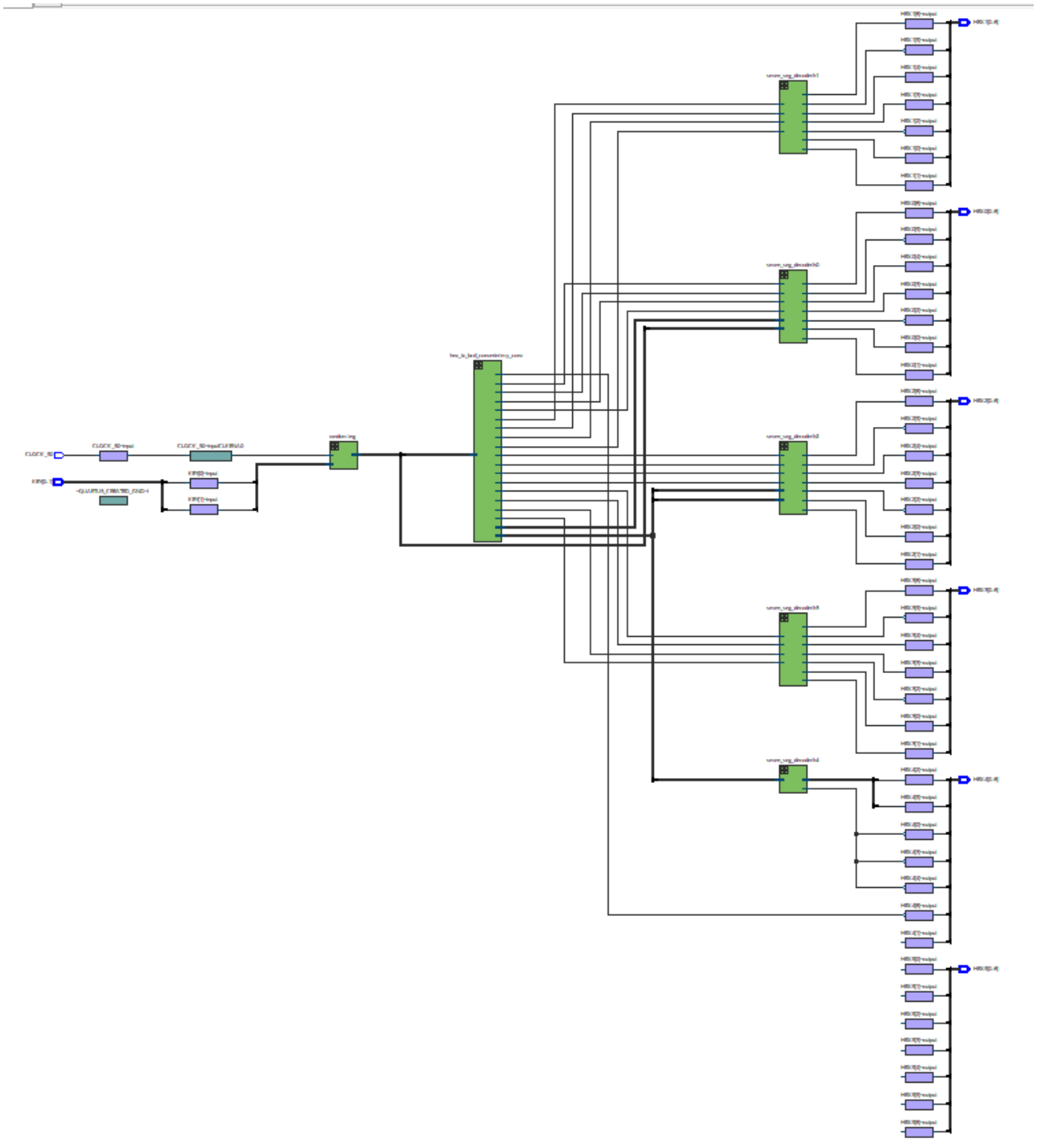


Figure 18. Technology map viewer (post-fitting) of the circuit.

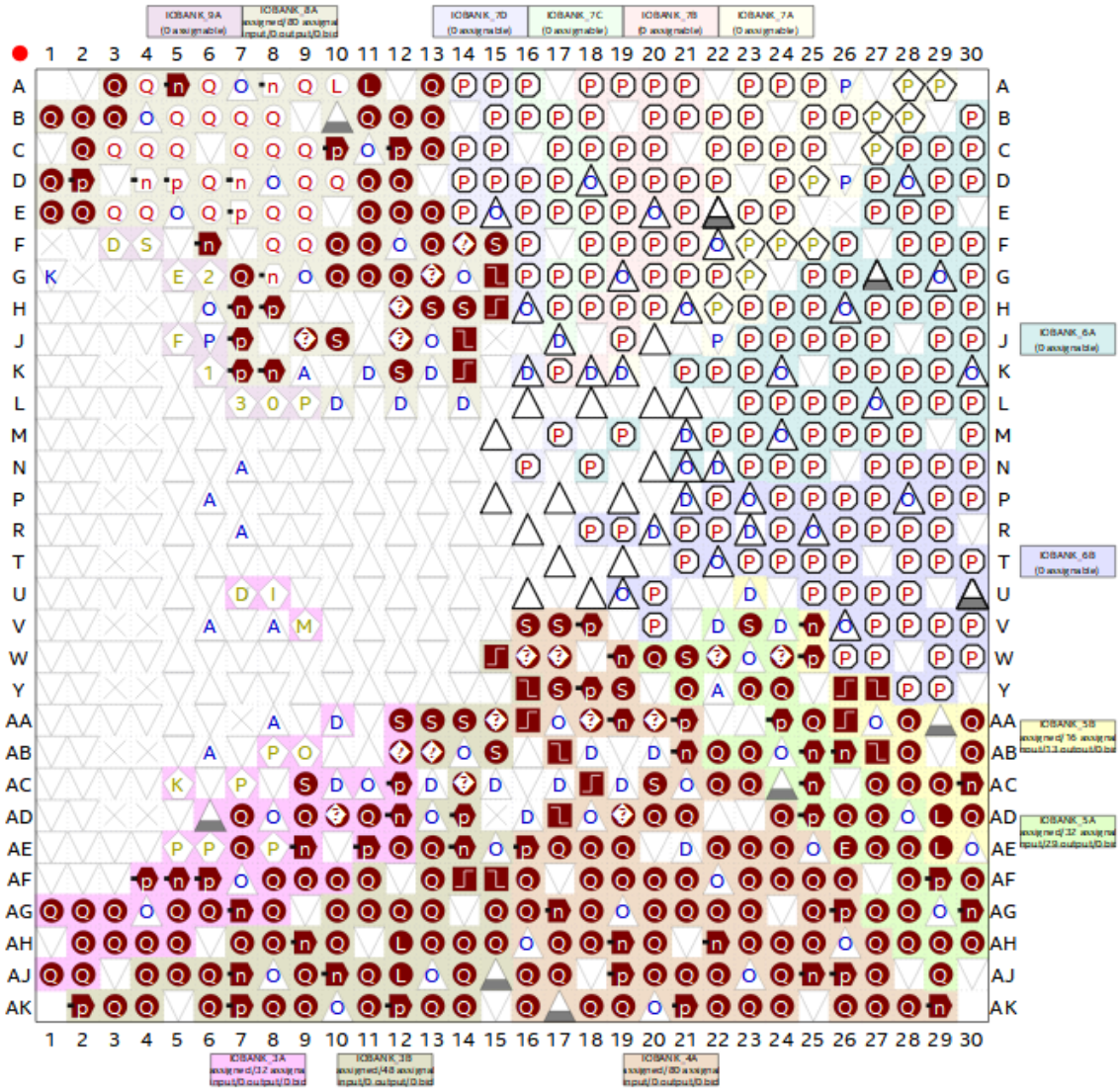


Figure 19a. Pin planner image.

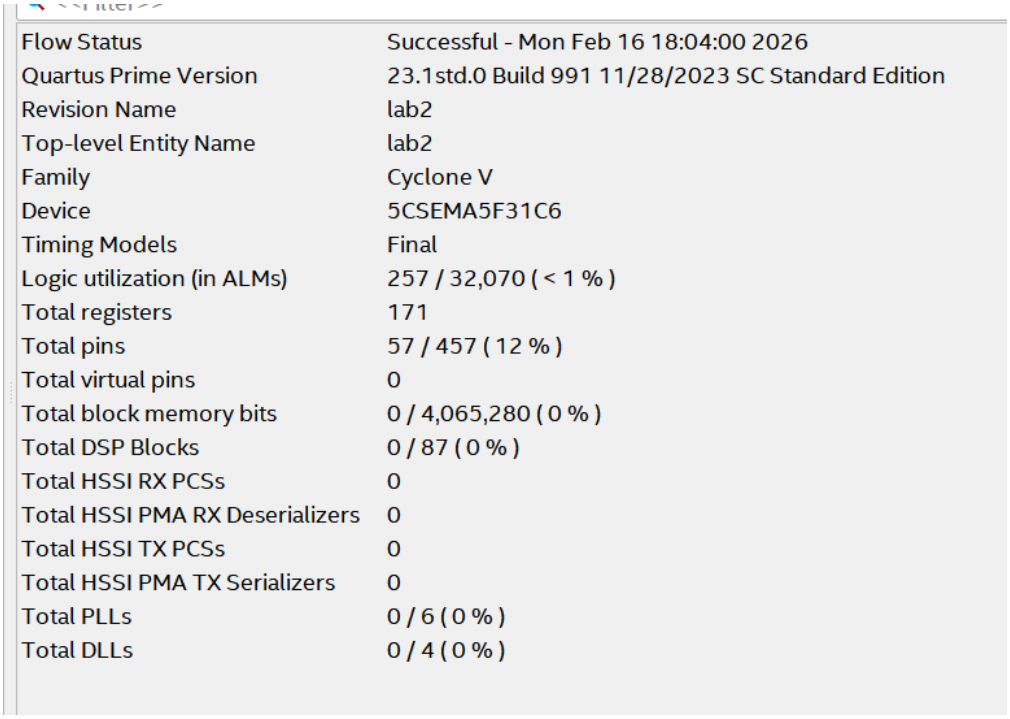
Node Name	Direction	Location	/O Banl	VREF Group	Fitter Location	I/O Standard	:serve	Current Strength	Slew Rate	Diffe
CLOCK_50	Input	PIN_AF14	3B	B3B_N0	PIN_AF14	2.5 V		12mA (default)		
HEX0[6]	Output	PIN_AH28	5A	B5A_N0	PIN_AH28	2.5 V		12mA (default)	1 (default)	
HEX0[5]	Output	PIN_AG28	5A	B5A_N0	PIN_AG28	2.5 V		12mA (default)	1 (default)	
HEX0[4]	Output	PIN_AF28	5A	B5A_N0	PIN_AF28	2.5 V		12mA (default)	1 (default)	
HEX0[3]	Output	PIN_AG27	5A	B5A_N0	PIN_AG27	2.5 V		12mA (default)	1 (default)	
HEX0[2]	Output	PIN_AE28	5A	B5A_N0	PIN_AE28	2.5 V		12mA (default)	1 (default)	
HEX0[1]	Output	PIN_AE27	5A	B5A_N0	PIN_AE27	2.5 V		12mA (default)	1 (default)	
HEX0[0]	Output	PIN_AE26	5A	B5A_N0	PIN_AE26	2.5 V		12mA (default)	1 (default)	
HEX1[6]	Output	PIN_AD27	5A	B5A_N0	PIN_AD27	2.5 V		12mA (default)	1 (default)	
HEX1[5]	Output	PIN_AF30	5A	B5A_N0	PIN_AF30	2.5 V		12mA (default)	1 (default)	
HEX1[4]	Output	PIN_AF29	5A	B5A_N0	PIN_AF29	2.5 V		12mA (default)	1 (default)	
HEX1[3]	Output	PIN_AG30	5A	B5A_N0	PIN_AG30	2.5 V		12mA (default)	1 (default)	
HEX1[2]	Output	PIN_AH30	5A	B5A_N0	PIN_AH30	2.5 V		12mA (default)	1 (default)	
HEX1[1]	Output	PIN_AH29	5A	B5A_N0	PIN_AH29	2.5 V		12mA (default)	1 (default)	
HEX1[0]	Output	PIN_AJ29	5A	B5A_N0	PIN_AJ29	2.5 V		12mA (default)	1 (default)	
HEX2[6]	Output	PIN_AC30	5B	B5B_N0	PIN_AC30	2.5 V		12mA (default)	1 (default)	
HEX2[5]	Output	PIN_AC29	5B	B5B_N0	PIN_AC29	2.5 V		12mA (default)	1 (default)	
HEX2[4]	Output	PIN_AD30	5B	B5B_N0	PIN_AD30	2.5 V		12mA (default)	1 (default)	
HEX2[3]	Output	PIN_AC28	5B	B5B_N0	PIN_AC28	2.5 V		12mA (default)	1 (default)	
HEX2[2]	Output	PIN_AD29	5B	B5B_N0	PIN_AD29	2.5 V		12mA (default)	1 (default)	
HEX2[1]	Output	PIN_AE29	5B	B5B_N0	PIN_AE29	2.5 V		12mA (default)	1 (default)	
HEX2[0]	Output	PIN_AB23	5A	B5A_N0	PIN_AB23	2.5 V		12mA (default)	1 (default)	
HEX3[6]	Output	PIN_AB22	5A	B5A_N0	PIN_AB22	2.5 V		12mA (default)	1 (default)	
HEX3[5]	Output	PIN_AB25	5A	B5A_N0	PIN_AB25	2.5 V		12mA (default)	1 (default)	
HEX3[4]	Output	PIN_AB28	5B	B5B_N0	PIN_AB28	2.5 V		12mA (default)	1 (default)	
HEX3[3]	Output	PIN_AC25	5A	B5A_N0	PIN_AC25	2.5 V		12mA (default)	1 (default)	
HEX3[2]	Output	PIN_AD25	5A	B5A_N0	PIN_AD25	2.5 V		12mA (default)	1 (default)	
HEX3[1]	Output	PIN_AC27	5A	B5A_N0	PIN_AC27	2.5 V		12mA (default)	1 (default)	
HEX3[0]	Output	PIN_AD26	5A	B5A_N0	PIN_AD26	2.5 V		12mA (default)	1 (default)	
HEX4[6]	Output	PIN_W25	5B	B5B_N0	PIN_W25	2.5 V		12mA (default)	1 (default)	
HEX4[5]	Output	PIN_V23	5A	B5A_N0	PIN_V23	2.5 V		12mA (default)	1 (default)	
HEX4[4]	Output	PIN_W24	5A	B5A_N0	PIN_W24	2.5 V		12mA (default)	1 (default)	
HEX4[3]	Output	PIN_W22	5A	B5A_N0	PIN_W22	2.5 V		12mA (default)	1 (default)	
HEX4[2]	Output	PIN_Y24	5A	B5A_N0	PIN_Y24	2.5 V		12mA (default)	1 (default)	
HEX4[1]	Output	PIN_Y23	5A	B5A_N0	PIN_Y23	2.5 V		12mA (default)	1 (default)	
HEX4[0]	Output	PIN_AA24	5A	B5A_N0	PIN_AA24	2.5 V		12mA (default)	1 (default)	
HEX5[6]	Output	PIN_AA25	5A	B5A_N0	PIN_AA25	2.5 V		12mA (default)	1 (default)	
HEX5[5]	Output	PIN_AA26	5B	B5B_N0	PIN_AA26	2.5 V		12mA (default)	1 (default)	
HEX5[4]	Output	PIN_AB26	5A	B5A_N0	PIN_AB26	2.5 V		12mA (default)	1 (default)	
HEX5[3]	Output	PIN_AB27	5B	B5B_N0	PIN_AB27	2.5 V		12mA (default)	1 (default)	
HEX5[2]	Output	PIN_Y27	5B	B5B_N0	PIN_Y27	2.5 V		12mA (default)	1 (default)	
HEX5[1]	Output	PIN_AA28	5B	B5B_N0	PIN_AA28	2.5 V		12mA (default)	1 (default)	
HEX5[0]	Output	PIN_V25	5B	B5B_N0	PIN_V25	2.5 V		12mA (default)	1 (default)	
KEY[1]	Input	PIN_AA15	3B	B3B_N0	PIN_AA15	2.5 V		12mA (default)		
KEY[0]	Input	PIN_AA14	3B	B3B_N0	PIN_AA14	2.5 V		12mA (default)		

Figure 19b. Pin planner table.

Part 2: Reaction Time Contest FSM

Top-Level Module

The Random, HEX to BCD Converter, and the Seven Segment Decoder modules were used for the Reaction Time Contest FSM from Lab 2 Part 1 above. The Counter, Mux (4-to-1), Clock Divider, and Blink HEX Led modules in Lab Tutorial 2 and Prelab 2 were also used for the Reaction Time Contest FSM. Please refer to the Lab Tutorial and Prelab 2 report for further information regarding those modules.

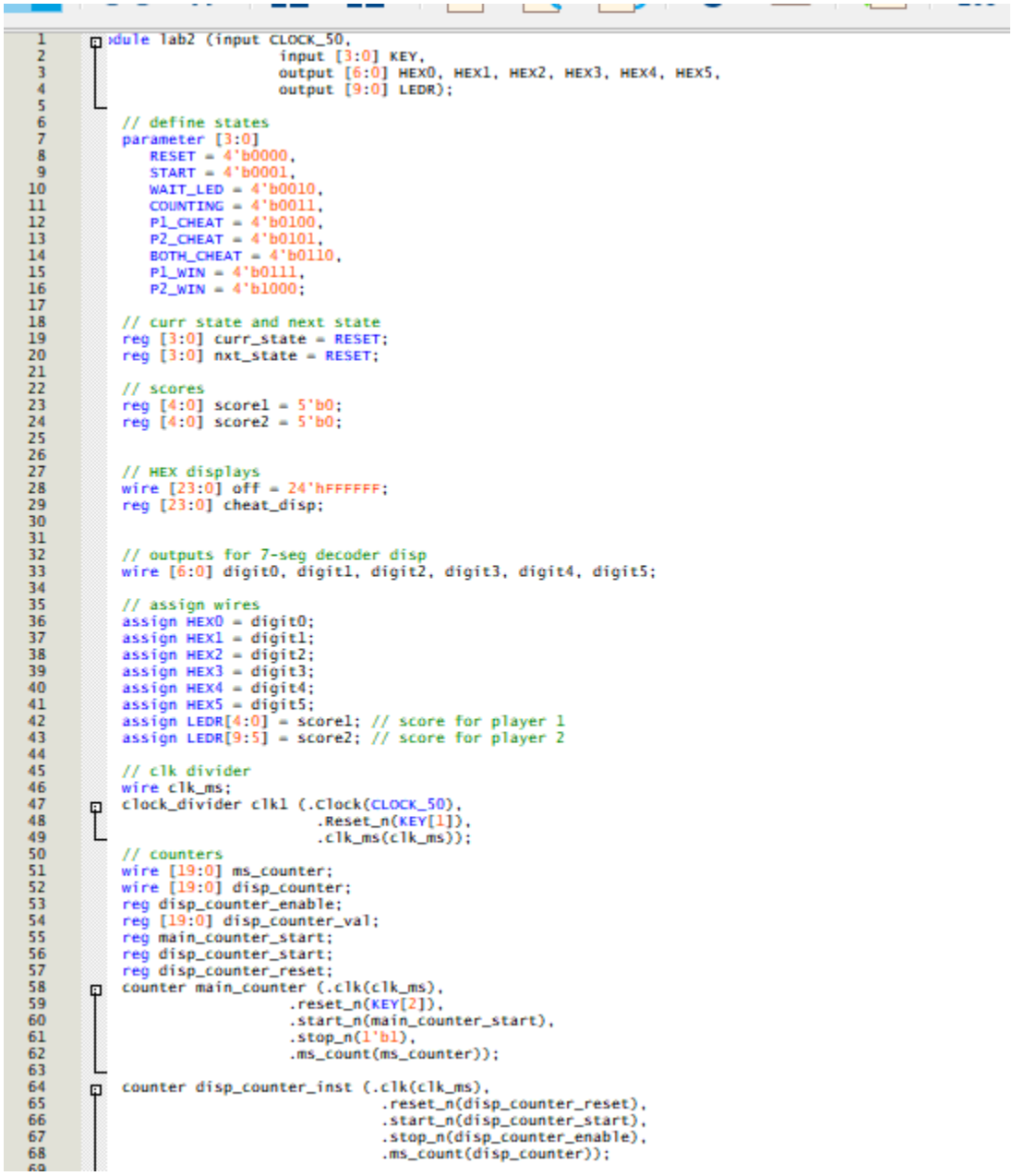


Flow Status	Successful - Mon Feb 16 18:04:00 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab2
Top-level Entity Name	lab2
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	257 / 32,070 (< 1 %)
Total registers	171
Total pins	57 / 457 (12 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 20. Compilation report of the Reaction Time Contest FSM module.

Table 5. Summary for Components for the Reaction Time Contest FSM in Lab 2 Part 2.

Feature	Quantity
Total number of logic elements	257
Total number of registers	171
Total number of pins	57
Max number of logic elements that can fit on FPGA	32,070



```
1   odule lab2 (input CLOCK_50,
2                   input [3:0] KEY,
3                   output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5,
4                   output [9:0] LEDR);
5
6     // define states
7     parameter [3:0]
8        RESET = 4'b0000,
9        START = 4'b0001,
10       WAIT_LED = 4'b0010,
11       COUNTING = 4'b0011,
12       P1_CHEAT = 4'b0100,
13       P2_CHEAT = 4'b0101,
14       BOTH_CHEAT = 4'b0110,
15       P1_WIN = 4'b0111,
16       P2_WIN = 4'b1000;
17
18    // curr state and next state
19    reg [3:0] curr_state = RESET;
20    reg [3:0] nxt_state = RESET;
21
22    // scores
23    reg [4:0] score1 = 5'b0;
24    reg [4:0] score2 = 5'b0;
25
26
27    // HEX displays
28    wire [23:0] off = 24'hFFFFFF;
29    reg [23:0] cheat_disp;
30
31
32    // outputs for 7-seg decoder disp
33    wire [6:0] digit0, digit1, digit2, digit3, digit4, digit5;
34
35    // assign wires
36    assign HEX0 = digit0;
37    assign HEX1 = digit1;
38    assign HEX2 = digit2;
39    assign HEX3 = digit3;
40    assign HEX4 = digit4;
41    assign HEX5 = digit5;
42    assign LEDR[4:0] = score1; // score for player 1
43    assign LEDR[9:5] = score2; // score for player 2
44
45    // clk divider
46    wire clk_ms;
47    clock_divider clk1 (.Clock(CLOCK_50),
48                        .Reset_n(KEY[1]),
49                        .clk_ms(clk_ms));
50    // counters
51    wire [19:0] ms_counter;
52    wire [19:0] disp_counter;
53    reg disp_counter_enable;
54    reg [19:0] disp_counter_val;
55    reg main_counter_start;
56    reg disp_counter_start;
57    reg disp_counter_reset;
58    counter main_counter (.clk(clk_ms),
59                          .reset_n(KEY[2]),
60                          .start_n(main_counter_start),
61                          .stop_n(1'b1),
62                          .ms_count(ms_counter));
63
64    counter disp_counter_inst (.clk(clk_ms),
65                               .reset_n(disp_counter_reset),
66                               .start_n(disp_counter_start),
67                               .stop_n(disp_counter_enable),
68                               .ms_count(disp_counter));
```

Figure 21. Screenshot of the Reaction Time Contest FSM (full code in Code Section of report).

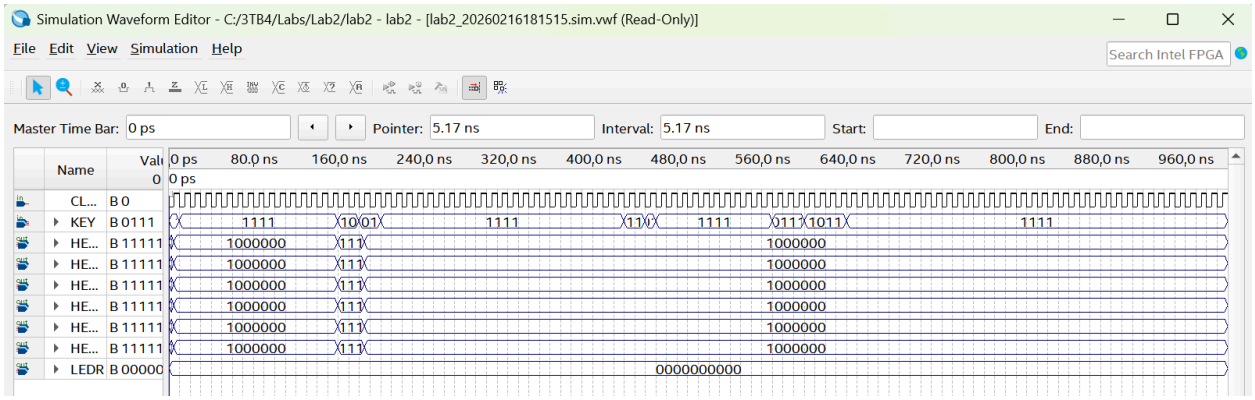


Figure 22. Simulation results for the Reaction Time Contest FSM.

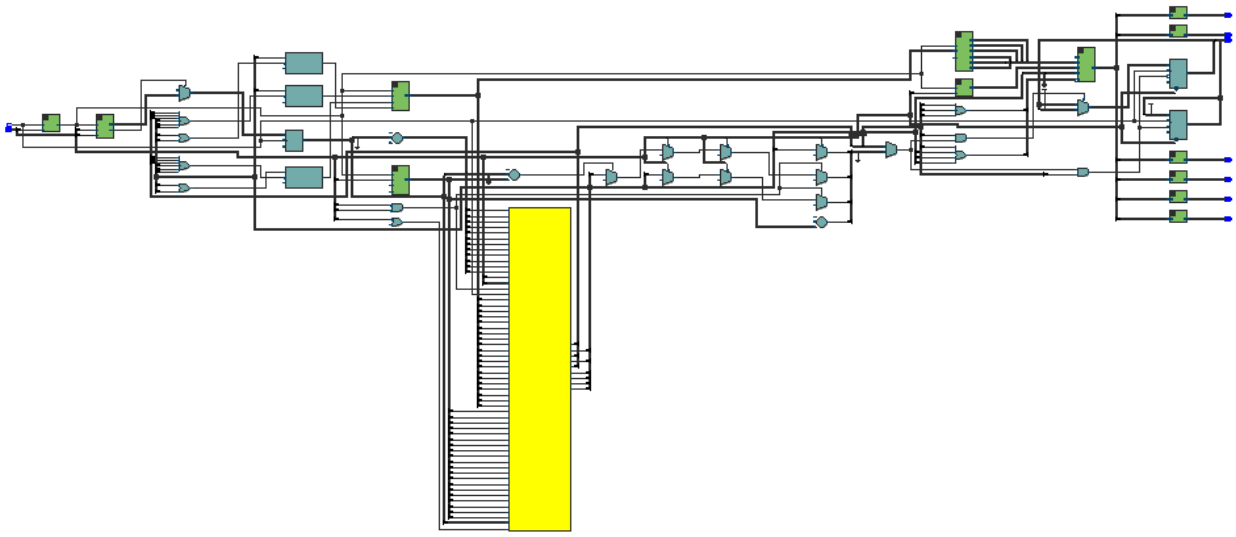


Figure 23. RTL view of the circuit.

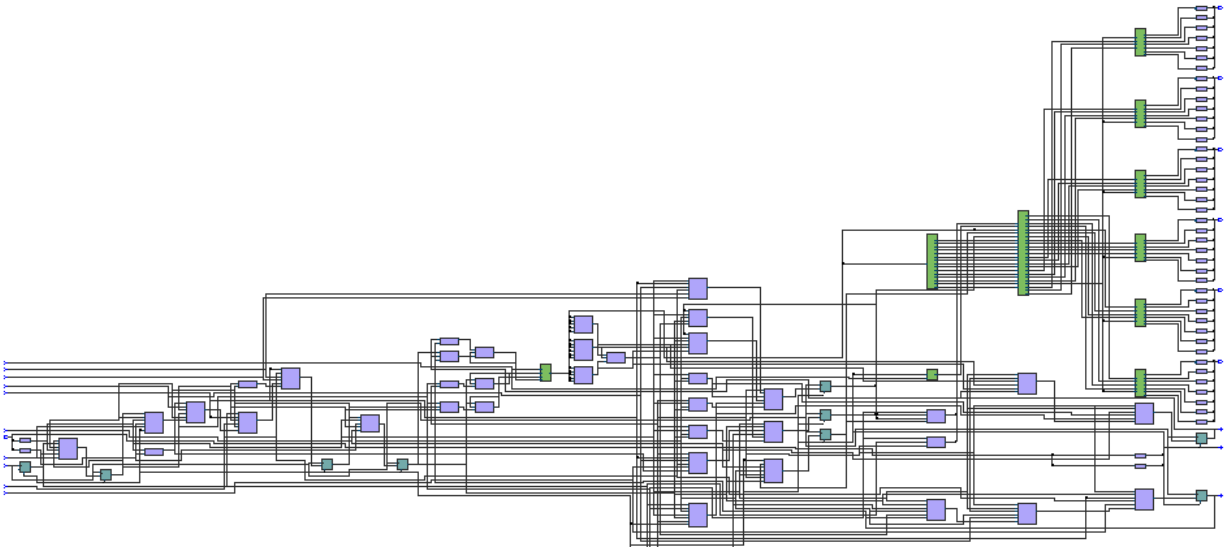


Figure 24. Technology map viewer (post-mapping) of the circuit.

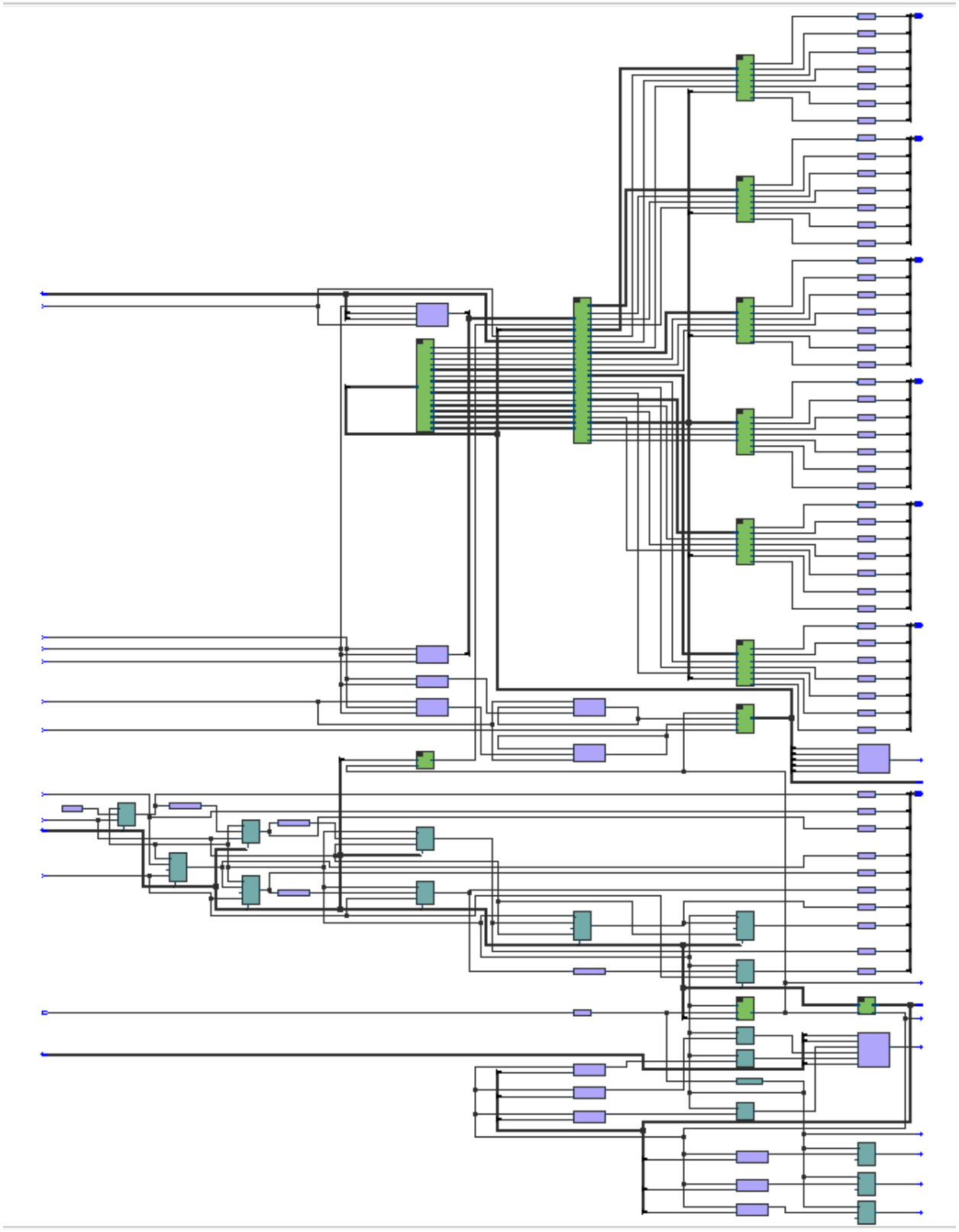


Figure 25. Technology map viewer (post-fitting) of the circuit.

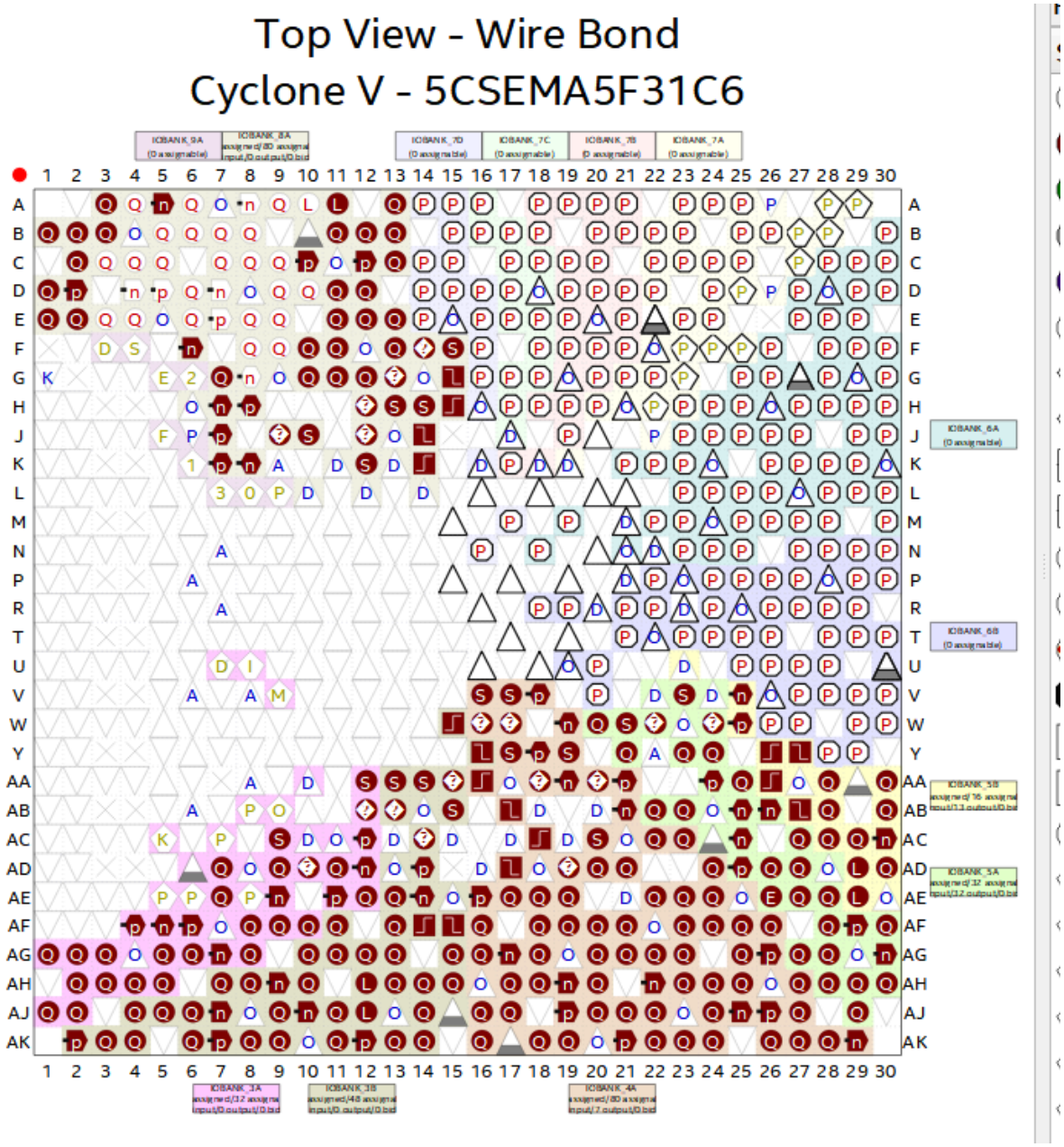


Figure 26a. Pin planner image.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserv
HEX0[6]	Output	PIN_AH28	5A	B5A_N0	PIN_AH28	2.5 V	
HEX0[5]	Output	PIN_AG28	5A	B5A_N0	PIN_AG28	2.5 V	
HEX0[4]	Output	PIN_AF28	5A	B5A_N0	PIN_AF28	2.5 V	
HEX0[3]	Output	PIN_AG27	5A	B5A_N0	PIN_AG27	2.5 V	
HEX0[2]	Output	PIN_AE28	5A	B5A_N0	PIN_AE28	2.5 V	
HEX0[1]	Output	PIN_AE27	5A	B5A_N0	PIN_AE27	2.5 V	
HEX0[0]	Output	PIN_AE26	5A	B5A_N0	PIN_AE26	2.5 V	
HEX1[6]	Output	PIN_AD27	5A	B5A_N0	PIN_AD27	2.5 V	
HEX1[5]	Output	PIN_AF30	5A	B5A_N0	PIN_AF30	2.5 V	
HEX1[4]	Output	PIN_AF29	5A	B5A_N0	PIN_AF29	2.5 V	
HEX1[3]	Output	PIN_AG30	5A	B5A_N0	PIN_AG30	2.5 V	
HEX1[2]	Output	PIN_AH30	5A	B5A_N0	PIN_AH30	2.5 V	
HEX1[1]	Output	PIN_AH29	5A	B5A_N0	PIN_AH29	2.5 V	
HEX1[0]	Output	PIN_AJ29	5A	B5A_N0	PIN_AJ29	2.5 V	
HEX2[6]	Output	PIN_AC30	5B	B5B_N0	PIN_AC30	2.5 V	
HEX2[5]	Output	PIN_AC29	5B	B5B_N0	PIN_AC29	2.5 V	
HEX2[4]	Output	PIN_AD30	5B	B5B_N0	PIN_AD30	2.5 V	
HEX2[3]	Output	PIN_AC28	5B	B5B_N0	PIN_AC28	2.5 V	
HEX2[2]	Output	PIN_AD29	5B	B5B_N0	PIN_AD29	2.5 V	
HEX2[1]	Output	PIN_AE29	5B	B5B_N0	PIN_AE29	2.5 V	
HEX2[0]	Output	PIN_AB23	5A	B5A_N0	PIN_AB23	2.5 V	
HEX3[6]	Output	PIN_AB22	5A	B5A_N0	PIN_AB22	2.5 V	
HEX3[5]	Output	PIN_AB25	5A	B5A_N0	PIN_AB25	2.5 V	
HEX3[4]	Output	PIN_AB28	5B	B5B_N0	PIN_AB28	2.5 V	
HEX3[3]	Output	PIN_AC25	5A	B5A_N0	PIN_AC25	2.5 V	
HEX3[2]	Output	PIN_AD25	5A	B5A_N0	PIN_AD25	2.5 V	
HEX3[1]	Output	PIN_AC27	5A	B5A_N0	PIN_AC27	2.5 V	
HEX3[0]	Output	PIN_AD26	5A	B5A_N0	PIN_AD26	2.5 V	
HEX4[6]	Output	PIN_W25	5B	B5B_N0	PIN_W25	2.5 V	
HEX4[5]	Output	PIN_V23	5A	B5A_N0	PIN_V23	2.5 V	
HEX4[4]	Output	PIN_W24	5A	B5A_N0	PIN_W24	2.5 V	
HEX4[3]	Output	PIN_W22	5A	B5A_N0	PIN_W22	2.5 V	
HEX4[2]	Output	PIN_Y24	5A	B5A_N0	PIN_Y24	2.5 V	
HEX4[1]	Output	PIN_Y23	5A	B5A_N0	PIN_Y23	2.5 V	
HEX4[0]	Output	PIN_AA24	5A	B5A_N0	PIN_AA24	2.5 V	
HEX5[6]	Output	PIN_AA25	5A	B5A_N0	PIN_AA25	2.5 V	
HEX5[5]	Output	PIN_AA26	5B	B5B_N0	PIN_AA26	2.5 V	
HEX5[4]	Output	PIN_AB26	5A	B5A_N0	PIN_AB26	2.5 V	
HEX5[3]	Output	PIN_AB27	5B	B5B_N0	PIN_AB27	2.5 V	
HEX5[2]	Output	PIN_Y27	5B	B5B_N0	PIN_Y27	2.5 V	
HEX5[1]	Output	PIN_AA28	5B	B5B_N0	PIN_AA28	2.5 V	
HEX5[0]	Output	PIN_V25	5B	B5B_N0	PIN_V25	2.5 V	
KEY[3]	Input	PIN_Y16	3B	B3B_N0	PIN_Y16	2.5 V	
KEY[2]	Input	PIN_W15	3B	B3B_N0	PIN_W15	2.5 V	
KEY[1]	Input	PIN_AA15	3B	B3B_N0	PIN_AA15	2.5 V	
KEY[0]	Input	PIN_AA14	3B	B3B_N0	PIN_AA14	2.5 V	
LEDR[9]	Output	PIN_Y21	5A	B5A_N0	PIN_Y21	2.5 V	
LEDR[8]	Output	PIN_W21	5A	B5A_N0	PIN_W21	2.5 V	
LEDR[7]	Output	PIN_W20	5A	B5A_N0	PIN_W20	2.5 V	
LEDR[6]	Output	PIN_Y19	4A	B4A_N0	PIN_Y19	2.5 V	
LEDR[5]	Output	PIN_W19	4A	B4A_N0	PIN_W19	2.5 V	
LEDR[4]	Output	PIN_W17	4A	B4A_N0	PIN_W17	2.5 V	
LEDR[3]	Output	PIN_V18	4A	B4A_N0	PIN_V18	2.5 V	
LEDR[2]	Output	PIN_V17	4A	B4A_N0	PIN_V17	2.5 V	
LEDR[1]	Output	PIN_W16	4A	B4A_N0	PIN_W16	2.5 V	
LEDR[0]	Output	PIN_V16	4A	B4A_N0	PIN_V16	2.5 V	

Figure 26b. Pin planner table.

Code

Part 1: Random Number Generator (RNG)

Random

```
module random (input clk, reset_n, resume_n,
                output reg [13:0] random,
                output reg rnd_ready);
  //for 14 bits Liner Feedback Shift Register,
  //the Taps that need to be XNORed or XORed are: 14, 5, 3, 1
  wire xnor_taps, and_allbits, feedback;
  reg [13:0] reg_values;
  reg enable = 1;

  always @ (posedge clk, negedge reset_n, negedge resume_n) begin
    if (!reset_n) begin
      reg_values <= 14'b11111111111111;
      //the LFSR cannot be all 0 at beginning.
      enable <= 1;
      rnd_ready <= 0;
    end

    else if (!resume_n) begin
      enable <= 1;
      rnd_ready <= 0;
      reg_values <= reg_values;
    end

    else begin
      if (enable) begin
        reg_values[13] <= reg_values[0];
        reg_values[12:5] <= reg_values[13:6];
        reg_values[4] <= reg_values[0] ^ reg_values[5];
        // tap 5 of the diagram from the lab manual
        reg_values[3] <= reg_values[4];
        reg_values[2] <= reg_values[0] ^ reg_values[3];
        // tap 3 of the diagram from the lab manual
        reg_values[1] <= reg_values[2];
        reg_values[0] <= reg_values[0] ^ reg_values[1];
        // tap 1 of the diagram from the lab manual

        // confirm sure the random number is between 1000 and 5000
        if (reg_values <= 5000 && reg_values >= 1000) begin
          rnd_ready <= 1'b1;
          enable <= 1'b0;
          random <= reg_values;
        end
```

```
            else begin
                rnd_ready <= 1'b0;
            end
        end //end of ENABLE.
    end
  end
endmodule
```

HEX to BCD Converter

```
// CONVERTS HEX TO BCD

module hex_to_bcd_converter(input wire clk, reset,
                                                    input wire [19:0]
hex_number,
                                                    output [3:0]
bcd_digit_0,bcd_digit_1,bcd_digit_2,bcd_digit_3, bcd_digit_4, bcd_digit_5);
        // DE1-SoC has 6 7_seg_LEDs, 20 bits can represent decimal 999999.
        //This module is designed to convert a 20 bit binary representation to BCD
        integer i, k, j;
        wire [19:0] hex_number1; // the last 20 bits of input hex_number
        reg [3:0] bcd_digit [5:0]; // 6 BCD for 0-5
        assign hex_number1 = hex_number[19:0];

        assign bcd_digit_0 = bcd_digit[0];
        assign bcd_digit_1 = bcd_digit[1];
        assign bcd_digit_2 = bcd_digit[2];
        assign bcd_digit_3 = bcd_digit[3];
        assign bcd_digit_4 = bcd_digit[4];
        assign bcd_digit_5 = bcd_digit[5];

        always @ (*) begin
                if (!reset) begin
                        //set all 6 digits to 0
                        for (j=5 ; j>=0 ; j=j-1) begin
                                bcd_digit[j] = 4'b0000;
                        end
                end
                //set all 6 digits to 0
                for (j=5 ; j>=0 ; j=j-1) begin
                        bcd_digit[j] = 4'b0000;
                end

                //shift 20 times
                for (i=19; i>=0; i=i-1)
                begin

                        //check all 6 BCD tetrads
                        for (k=5; k>=0; k=k-1)
                        begin
                                // if >=5 then add 3
                                if (bcd_digit[k] >= 5) begin
```

```
                        bcd_digit[k] = bcd_digit[k] + 4'b0011;
                end
        end

        //shift one bit of BIN/HEX left
        //for the first 5 tetrads
        for (k=5; k>=1; k=k-1)
        begin
                bcd_digit[k]=bcd_digit[k] << 1;
                bcd_digit[k][0]=bcd_digit[k-1][3];
        end

        //shift one bit of BIN/HEX left, for the last tetrad
        /* fill your code here */
        bcd_digit[0] = bcd_digit[0] << 1;
        bcd_digit[0][0] = hex_number[i];

    end //end for loop
  end //end of always.
endmodule
```

Seven-Segment Decoder

```
module seven_seg_decoder(input [3:0] x, output [6:0] hex_LEDs);
   reg [6:0] reg_LEDs;

   assign hex_LEDs[0] = (~x[3] & ~x[2] & ~x[1] & x[0]) |
                        (x[2] & ~x[1] & ~x[0]) |
                        (x[3] & x[2] & x[1]);
   assign hex_LEDs[1] = (~x[3] & x[2] & ~x[1] & x[0]) |
                        (~x[3] & x[2] & x[1] & ~x[0]) |
                        (x[3] & x[2] & ~x[1] & ~x[0]) |
                        (x[3] & x[1] & x[0]);

   assign hex_LEDs[6:2]=reg_LEDs[6:2];

   always @(*)
   begin
      case (x)
         4'b0000: reg_LEDs[6:2]=5'b10000; //7'b1000000 decimal 0
         4'b0001: reg_LEDs[6:2]=5'b11110; //7'b1111001 decimal 1
         4'b0010: reg_LEDs[6:2]=5'b01001; //7'b0100100 decimal 2
         4'b0011: reg_LEDs[6:2]=5'b01100; //7'b0110000 decimal 3
         4'b0100: reg_LEDs[6:2]=5'b00110; //7'b0011001 decimal 4
         4'b0101: reg_LEDs[6:2]=5'b00100; //7'b0010010 decimal 5
         4'b0110: reg_LEDs[6:2]=5'b00000; //7'b0000010 decimal 6
         4'b0111: reg_LEDs[6:2]=5'b11110; //7'b1111000 decimal 7
         4'b1000: reg_LEDs[6:2]=5'b00000; //7'b0000000 decimal 8
         4'b1001: reg_LEDs[6:2]=5'b00100; //7'b0010000 decimal 9
                                  4'b1010: reg_LEDs[6:2]=5'b01000; //7'b0001000 A
                                  4'b1011: reg_LEDs[6:2]=5'b00011; //7'b0000011 b
                                  4'b1100: reg_LEDs[6:2]=5'b00110; //7'b1000110 C
                                  4'b1101: reg_LEDs[6:2]=5'b00001; //7'b0100001 d
                                  4'b1110: reg_LEDs[6:2]=5'b00110; //7'b0000110 E
         4'b1111: reg_LEDs[6:2]=5'b11111; //7'b1111111 OFF
         default: reg_LEDs[6:2] = 5'b11111;

      endcase
   end
endmodule
```

Top Level Module

```
module lab2_rng (
   input CLOCK_50,
   input [1:0] KEY,      // KEY[0] for Reset, KEY[1] for Resume
   output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5
);

        wire [13:0] rand_num;
        wire rnd_ready;
        wire [3:0] bcd0, bcd1, bcd2, bcd3, bcd4, bcd5;

        // random number generator
        random rng (.clk(CLOCK_50),
                                        .reset_n(KEY[0]),
                                        .resume_n(KEY[1]),
                                        .random(rand_num),
                                        .rnd_ready(rnd_ready));

   // binary to BCD converter
   hex_to_bcd_converter my_conv (.clk(CLOCK_50),

.reset(KEY[0]),

.hex_number({6'b0, rand_num}),

.bcd_digit_0(bcd0),

.bcd_digit_1(bcd1),

.bcd_digit_2(bcd2),

.bcd_digit_3(bcd3),

.bcd_digit_4(bcd4),

.bcd_digit_5(bcd5));

   // 7-segment decoder
   seven_seg_decoder h0 (.x(bcd0), .hex_LEDs(HEX0));
   seven_seg_decoder h1 (.x(bcd1), .hex_LEDs(HEX1));
   seven_seg_decoder h2 (.x(bcd2), .hex_LEDs(HEX2));
   seven_seg_decoder h3 (.x(bcd3), .hex_LEDs(HEX3));
   seven_seg_decoder h4 (.x(bcd4), .hex_LEDs(HEX4));
   seven_seg_decoder h5 (.x(bcd5), .hex_LEDs(HEX5));

endmodule
```

Part 2: FSM for Reaction Time Contest

Clock Divider

```
// CLOCK DIVIDER => MAKES FAST CLOCK (50 MHz) --> SLOWER (1 KHz)

module clock_divider (input Clock, Reset_n, output reg clk_ms);
	parameter factor = 50000; //50000; // 32'h000061a7;
	reg [31:0] countQ;

	always @ (posedge Clock, negedge Reset_n) begin
		if (!Reset_n) begin
			countQ <= 32'b0; // reset countQ to 0
			clk_ms <= 1'b0;
		end
		else begin
			if (countQ < factor/2) begin // clk is low
				countQ <= countQ + 1'b1; // increament countQ
				clk_ms <= 1'b1;
			end
			else if (countQ < factor) begin // clk is high
				countQ <= countQ + 1'b1;
				clk_ms <= 1'b0;

			end
			else begin //countQ == factor => nxt clk cycle
				countQ <= 32'b0; // reset countQ
			end
		end
	end
endmodule
```

Counter

```
// COUNTER

module counter(input clk, input reset_n, start_n, stop_n, output reg [19:0] ms_count);
	reg isCounting; // counter flag

	always @(posedge clk or negedge reset_n) begin
		if (!reset_n) begin
			ms_count <= 20'b0; // reset counter
			isCounting <= 1'b0; // reset flag
		end
		else begin
			if (!start_n) begin
				ms_count <= 20'b0;
				isCounting <= 1'b1; // start counting
			end
			else if (!stop_n) begin
				ms_count <= ms_count; // store prev value
				isCounting <= 1'b0; // stop counting
			end
			if (isCounting && stop_n)  begin
				ms_count <= ms_count + 1'b1;
			end
		end
	end
endmodule
```

Blink HEX LEDs

```
module blink_leds (input clk,
                                                input reset_n,
                                                output reg [23:0] LEDs);

        parameter factor = 200;
        reg [11:0] counter;

        always @(posedge clk or negedge reset_n) begin
                if (!reset_n) begin
                        counter <= 12'b0;
                        LEDs <= 24'b0;
                end

                else begin
                        if (counter < factor/2) begin
                                counter <= counter + 1'b1;
                        end
                        else if (counter < factor) begin
                                counter <= counter + 1'b1;
                                LEDs <= 24'b111111111111111111111111;
                        end
                        else begin
                                counter <= 12'b0;
                                LEDs <= 24'b0;
                        end
                end
        end
endmodule
```

Multiplexer (4-to-1)

```
module mux4_1(input [1:0] sel,
                                        input [23:0] a, b, c, d,
                                        output reg [23:0] out);

        always@(*) begin
                case (sel)
                        2'b00: out = a; // off
                        2'b01: out = b; //
                        2'b10: out = c;
                        2'b11: out = d;
                        default: out = 24'b0;
                endcase
        end

endmodule
```

Top Level Module

```
module lab2 (input CLOCK_50,
             input [3:0] KEY,
             output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5,
             output [9:0] LEDR);

        // define states
        parameter [3:0]
                RESET = 4'b0000,
                START = 4'b0001,
                WAIT_LED = 4'b0010,
                COUNTING = 4'b0011,
                P1_CHEAT = 4'b0100,
                P2_CHEAT = 4'b0101,
                BOTH_CHEAT = 4'b0110,
                P1_WIN = 4'b0111,
                P2_WIN = 4'b1000;

        // curr state and next state
        reg [3:0] curr_state = RESET;
        reg [3:0] nxt_state = RESET;

        // scores
        reg [4:0] score1 = 5'b0;
        reg [4:0] score2 = 5'b0;


        // HEX displays
        wire [23:0] off = 24'hFFFFFF;
        reg [23:0] cheat_disp;


        // outputs for 7-seg decoder disp
        wire [6:0] digit0, digit1, digit2, digit3, digit4, digit5;

        // assign wires
        assign HEX0 = digit0;
        assign HEX1 = digit1;
        assign HEX2 = digit2;
        assign HEX3 = digit3;
        assign HEX4 = digit4;
        assign HEX5 = digit5;
        assign LEDR[4:0] = score1; // score for player 1
        assign LEDR[9:5] = score2; // score for player 2

        // clk divider
        wire clk_ms;
        clock_divider clk1 (.Clock(CLOCK_50),
                                                        .Reset_n(KEY[1]),
                                                        .clk_ms(clk_ms));
```

```
	// counters
	wire [19:0] ms_counter;
	wire [19:0] disp_counter;
	reg disp_counter_enable;
	reg [19:0] disp_counter_val;
	reg main_counter_start;
	reg disp_counter_start;
	reg disp_counter_reset;
	counter main_counter (.clk(clk_ms),
                                                .reset_n(KEY[2]),
                                                .start_n(main_counter_start),
                                                .stop_n(1'b1),
                                                .ms_count(ms_counter));

	counter disp_counter_inst (.clk(clk_ms),
                .reset_n(disp_counter_reset),
                .start_n(disp_counter_start),
                .stop_n(disp_counter_enable),
                .ms_count(disp_counter));

	// RNG
	wire [13:0] random_wait;
	reg [13:0] random_wait_time;
	wire rng_ready;
	random rng1 (.clk(clk_ms),
                                            .reset_n(KEY[1]),
                                            .resume_n(KEY[2]),
        .random(random_wait),
        .rnd_ready(rng_ready));

	// HEX to BCD converter
	wire [23:0] bcd_out;
	hex_to_bcd_converter bcd1 (.clk(clk_ms),
                                                                .reset(reset_n),

.hex_number(disp_counter),

.bcd_digit_0(bcd_out[3:0]),

.bcd_digit_1(bcd_out[7:4]),

.bcd_digit_2(bcd_out[11:8]),

.bcd_digit_3(bcd_out[15:12]),

.bcd_digit_4(bcd_out[19:16]),

.bcd_digit_5(bcd_out[23:20]));

	// LEDs blinker
	wire [23:0] blink_output;
```

```
blink_leds blink1 (.clk(clk_ms),
	.reset_n(KEY[1]),
	.LEDs(blink_output));

// mux
wire [23:0] mux_out;
reg [1:0] sel;
mux4_1 mux(.sel(sel),
                                .a(off), //00
                                .b(bcd_out), //01
                                .c(blink_output), //10
                                .d(cheat_disp), //11
                                .out(mux_out));

// 7-seg disp decoder
seven_seg_decoder seg0 (.x(mux_out[3:0]), .hex_LEDs(digit0));
seven_seg_decoder seg1 (.x(mux_out[7:4]), .hex_LEDs(digit1));
seven_seg_decoder seg2 (.x(mux_out[11:8]), .hex_LEDs(digit2));
seven_seg_decoder seg3 (.x(mux_out[15:12]), .hex_LEDs(digit3));
seven_seg_decoder seg4 (.x(mux_out[19:16]), .hex_LEDs(digit4));
seven_seg_decoder seg5 (.x(mux_out[23:20]), .hex_LEDs(digit5));

// check if rng is ready
always @(posedge CLOCK_50) begin
	if (rng_ready) begin
		random_wait_time = random_wait;
	end
	else begin
		random_wait_time = 2000; // default value
	end
end

// FSM
always @(posedge CLOCK_50 or negedge KEY[1]) begin
	if (!KEY[1]) begin
		curr_state <= RESET;
	end
	else begin
		curr_state <= nxt_state;
	end
end

// determine nxt_state
always @(*) begin
	// default conditions
	nxt_state = curr_state;
	sel = 2'b00;
	cheat_disp = 24'b0;

	case(curr_state)
		RESET: begin
```

```
        //disp_counter_val = 20'b0;
        sel = 2'b00;
        nxt_state = START;
        disp_counter_enable = 0;
end

START: begin
        sel = 2'b10; // blink
        main_counter_start = 0;
        disp_counter_reset = 0;
        if (ms_counter >= 5000) begin
                nxt_state = WAIT_LED;
        end
        else begin
                nxt_state = START;
        end
end

WAIT_LED: begin
        sel = 2'b00; // off, waiting period
        disp_counter_reset = 0;
        // check cheating
        if (!KEY[0] && !KEY[3]) begin
                nxt_state = BOTH_CHEAT;
        end
        else if (!KEY[0]) begin
                nxt_state = P1_CHEAT;
        end
        else if (!KEY[3]) begin
                nxt_state = P2_CHEAT;
        end
        else if (ms_counter >= random_wait_time + 5000) begin
                nxt_state = COUNTING;
        end
end

// COUNTING state
COUNTING: begin
        sel = 2'b01; // display reaction time
        disp_counter_reset = 1;
        disp_counter_enable = 1; // start counting reaction
        disp_counter_start = 0;

        if (!KEY[0] && !KEY[3]) begin
                nxt_state = BOTH_CHEAT;
        end
        else if (!KEY[0]) begin
                if (disp_counter < 80) begin
                        nxt_state = P1_CHEAT;
                end
                else begin
```

```
                        nxt_state = P1_WIN;
                end
        end
        else if (!KEY[3]) begin
                if (disp_counter < 80) begin
                        nxt_state = P2_CHEAT;
                end
                else begin
                        nxt_state = P2_WIN;
                end
        end
end

P1_CHEAT: begin
        sel = 2'b11;
        cheat_disp = 24'h111111;
        disp_counter_enable = 0;
        if (!KEY[2]) begin
                nxt_state = WAIT_LED;
        end
end

P2_CHEAT: begin
        sel = 2'b11;
        cheat_disp = 24'h222222;
        disp_counter_enable = 0;
        if (!KEY[2]) begin
                nxt_state = WAIT_LED;
        end
end

BOTH_CHEAT: begin
        sel = 2'b11;
        cheat_disp = 24'h888888;
        disp_counter_enable = 0;
        if (!KEY[2]) begin
                nxt_state = WAIT_LED;
        end
end

P1_WIN: begin
        disp_counter_enable = 0;
        disp_counter_start = 1;
        sel = 2'b01; // display winning time
        if (!KEY[2]) begin
                nxt_state = WAIT_LED;
        end
end

P2_WIN: begin
        disp_counter_enable = 0;
```

```
                        disp_counter_start = 1;
                        sel = 2'b01; // display winning time
                        if (!KEY[2]) begin
                                nxt_state = WAIT_LED;
                        end
                end

                default: nxt_state = RESET;
        endcase
    end

    // update scores
    always @(posedge CLOCK_50 or negedge KEY[1]) begin
        if (!KEY[1]) begin
            score1 <= 5'b0;
            score2 <= 5'b0;
        end
        else begin
            if (curr_state == P1_WIN && nxt_state == WAIT_LED) begin
                score1 <= (score1 << 1) | 5'b00001;
            end
            else if (curr_state == P2_WIN && nxt_state == WAIT_LED) begin
                score2 <= (score2 >> 1) | 5'b10000;
            end
        end
    end

endmodule
```